# Hybris

# Coding Standards Part - 1

# Document History

| Version | Reason for change | Date | Modified by | Reviewed By |
|---|---|---|---|---|
| 1.0 | Initial Draft | September 13, 2017 | Anurag Mundra | Srinivas Jayanna |
| 1.1 | Formatting changes | November 14, 2017 | Anurag Mundra | Srinivas Jayanna |
| | | | | |

# Table of Contents

Coding Standards ..... 4
1 Naming Conventions ..... 4
1.1 Naming Convention in Hybris Commerce OOTB Code: ..... 4
1.2 Naming Conventions for Implementations ..... 5
1.2.1 Java ..... 5
1.2.2 Services/Facades ..... 6
1.2.3 Controllers ..... 6
1.2.4 Interceptors ..... 7
1.2.5 Dynamic Attribute Handlers ..... 7
1.2.6 CronJob ..... 7
1.2.7 Spring Beans ..... 7
2 Design principles in Hybris Commerce: ..... 8
2.1 Facades ..... 8
2.2 Services ..... 8
2.3 DAO ..... 9
2.4 General Design Pattern ..... 9
Where to See Examples of Those Principles ..... 9
3 Code Style ..... 10
3.1 Coding Principles ..... 10
3.2 Use of Null (Example of Coding Principle) ..... 10
3.2.1 Use of Null in Our Own Code ..... 10
4 Exception Handling ..... 11
4.1 Exception Handling (Example of Coding Principle) ..... 11
4.1.1 Validation of Input Parameters ..... 11
4.1.2 Validation of Results ..... 12
5 Code Style ..... 12
5.1 Optimizations vs. Readability ..... 12
5.2 Code formatting ..... 13
5.3 Javadoc ..... 14
5.4 *-items.xml ..... 14
5.4.1 Guidelines Inside of *-items.xml ..... 14
6 Tools ..... 14
6.1 Development IDE (e.g. Eclipse) ..... 15
6.2 PMD ..... 15
6.3 Sonar ..... 16

6.4 Findbugs ..... 16
6.5 Checkstyle ..... 17
7 Source Code Repository Exclusions ..... 17
Recommendations for Extensions ..... 18
1 Project Specific Extensions ..... 18
1.1 Project-Specific Prefix ..... 18
1.2 Extensions Location ..... 18
1.3 Reduced Template Concept ..... 18
1.4 Core Extensions ..... 19
1.4.1 Type Codes Ranges ..... 19
1.5 Project Data Extension ..... 19
1.6 Test Data Extension ..... 19
1.7 Other Project Extensions ..... 20
1.8 hMC Extension ..... 20
1.9 Backoffice Extension ..... 20
1.10 Integration Extensions ..... 20
2 IDE Configuration ..... 21
2.1 Working Sets/Module Groups ..... 21
2.2 Working Sets in Eclipse ..... 21
2.3 Module Groups in IntelliJ IDEA ..... 21
3 Wiki Page with Guidelines ..... 22
References ..... 22

# Coding Standards

The coding standards are not based only on Java language coding standards. They are also enriched with Hybris Commerce specific guidelines and restrictions which are more specific to the Hybris Commerce platform, in the following areas:

- Type system
- Extension naming
- Java coding standards and naming
- Code formatting
- Design principles

All custom code should follow the same standard. It is important to document the coding standard and make it available for all project participants.

The goal of these coding standards is to create consistent code and design through all developer and architects. With the right commitment of everybody, it is possible to come as close as possible to this goal. But every developer will still have its own signature in the result, which is fine.

The current coding guidelines should also be included in the code review process and the CI build. Every reviewer in the code review process should also spend some time to verify the newly created code matches the coding standard. It is a common issue that these standards will not be enforced when the projects come closer to a timeline. But to follow and enforce the coding standard is not only a task for the developer during creating of new code, it is also required that it is part of every refactoring. Including required changes in a normal refactoring process has the benefit of making the code “cleaner” without additional effort.

This goal can be supported by a various style of static code analysis tools, like PMD, FindBug, Checkstyle and so on, which has plugins in all major IDEs like Eclipse and IDEA Intellij.

Coding standards and conventions should be agreed upon by the whole team, we shouldn’t try to enforce standards in a top-down manner because these standards will never be really met. The development and architecture team should agree to it and document them. It is important to document the standards, which will make it easier for developers and architects to refer to during different phases of the project, including subsequent releases. The documentation helps in preventing a break in standards in later version of the implementation.

# 1 Naming Conventions

## 1.1 Naming Convention in Hybris Commerce OOTB Code:

| Area | Description |
|---|---|
| Extensions | Extension names should only contain lower-case letters and be named according to their content. It is also common to use a project-specific prefix for all extensions of the same project. |
| Packages | Packages should be indicating the area of their content, like core, storefront or initial data. In the case of Service Layer classes, which have an interface and an implementation, both should be packaged accordingly in separate packages. |
| Classes | Classes should use descriptive names, preferably in English. The name should begin with a capital letter as well as every beginning letter in the word. All other letters besides the beginning letter in a word should be lowercase.<br>Hybris Commerce implementations of interfaces are named with the following pattern<br>Default<Name of Interface> implements <Name of Interface><br>e.g. DefaultProductService implements ProductService |

# 1.2 Naming Conventions for Implementations

The definition of a coherent naming convention is a step often neglected in a project. However, it contributes to the code quality and it will be easier for a developer to maintain the code from another one if such a convention is rigorously observed. Hybris uses this naming convention and generally encourages its partners to follow the same practice.

## 1.2.1 Java

Hybris follows the standard Java naming convention of CamelCase for class, method, variable and constant names. This standard was established by Sun Microsystems and other Java communities. Below are examples of each:

| Identifier | Naming Convention | Example |
|---|---|---|
| Class | Class names are declared in UpperCamelCase, with the first letter of each word capitalized.<br>Class names should avoid using acronyms and abbreviations if at all possible. | public class MyClass() {...} |
| Method | Method declarations follow a similar naming convention to Classes; however, the first letter of the method name is NOT capitalized and follow lowerCamelCase notation. | public void myMethod() {...} |

| Identifier | Naming Convention | Example |
|---|---|---|
| Variable | Variable names also follow the lowerCamelCase notation. Variable names should have meaning and describe what it is being used for. You may declare variable with one character when it is to be used as a temporary variable. | int myVariable; |
| Constant | Constants are declared in all Upper-Case characters and words are separated by underscores ("_")<br>Constants may contain numbers/digits, however, not as the first character. | MY_CONSTANT |

### 1.2.2 Services/Facades

- Suffix the class name by Service or Facade
- Replace the prefix Default by a meaningful word describing the implementation characteristics or at least with Custom for overloaded
- Prefix the expended interfaces by the project name
- Combine the last two rules for expended interface implementations

**Example**

```
public interface AcmeProductService extends ProductService {...}

public class CustomAcmeProductService extends DefaultProductService implements
AcmeProductService {...}
```

### 1.2.3 Controllers

- The CMSComponents controllers must have the same name as the CMSComponents suffixed by Controller
- Controllers names must be suffixed by PageController when it generates only one page and only by Controller otherwise the prefix must be meaningful in comparison to the implemented functionality.

**Example**

```
public class NavigationBarComponentController {...}

public class CategoryPageController {...}

public class PaymentAutoResponseController {...}
```

### 1.2.4 Interceptors

- Prefix the Interceptor name by the entity to which it applies the most, eventually a token indicating the purpose of the Interceptor
- Suffix the Interceptor name by the interface it implements

**Example**

```
public class AccountBalanceValidateInterceptor {...}

public class AccountUniqueValidateInterceptor {...}

public class CronJobInitDefaultsInterceptor {...}
```

### 1.2.5 Dynamic Attribute Handlers

- Prefix the DynamicAttributeHandler's name by the item's name to which it applies.
- Follow this prefix by the attribute name
- Suffix the name by AttributeHandler

**Example**

```
public class TaskExpirationDateAttributeHandler {...}
```

### 1.2.6 CronJob

- Suffix the item's name by CronJob
- Replace CronJob by JobPerformable for implementation.

**Example**

```
public class ErpProductImportCronJob {...}

public class ErpProductImportJobPerformable {...}
```

### 1.2.7 Spring Beans

The Hybris platform uses the Spring Framework extensively and therefore declares a lot of Spring managed beans in XML configuration files. It is another common practice to name these Spring Beans in lowerCamelCase, with the first character of the bean ID being lower case.

**Example**

```
<alias alias="productService" name="myProductService" />
<bean id="myProductService"
class="com.mycompany.core.service.impl.MyProductServiceImpl"
parent="defaultProductService">
  <property name="myProductDao" ref="myProductDao" />
</bean>
```

# 2 Design principles in Hybris Commerce:

Hybris Commerce platform and accelerator code make heavy usage of the Spring framework. To leverage the Spring framework consistently inside of a custom project it is highly recommended to develop against interfaces. For each newly created facade, service and data access object (DAO), an interface should exist and every development should only be performed against it. This makes the code easier to test and to swap against another implementation.

## 2.1 Facades

A facade is a software design pattern that abstracts from an underlying implementation and offers an alternate, often reduced and less complex interface.

A facade is used to expose business logic to the front-end controllers and should only use facades, services, converter, populator, and data transfer objects (DTO) (the latter are actually named *Data in hybris platform). The usage of DTO's allows to reduce the number of exposed attributes and also prevents the usage of models in the presentation layer. The DTO itself should only contain values and no business logic.

A facade should never expose a model class because the usage of models will destroy the loose coupling of the UI to the business logic.

## 2.2 Services

The purpose of a service is to cover the business logic. All services together represent a public API. The service layer should be used to perform any kind of business logic, which will create, update or remove models. For self-created or extended services, it is recommended to start the name with the project name as prefix to make it visible that this service contains non-hybris OOTB code.

Example:

| | **Package and Class Name** |
|---|---|
| Interface | de.hybris.platform.order.CalculationService |
| OOTB implementation as<br><br>Accelerator service | de.hybris.platform.order.impl.DefaultCalculationService |
| custom implementation which extends/replaces the OOTB implementation | com.<project name>.cscockpit.order.impl.Custom<project name>CalculationService |

## 2.3 DAO

The usage of data access objects (DAO) is a good practice to abstract and encapsulate access to the data objects. The DAO can contain multiple methods for typical CRUD operations. For a Hybris Commerce project, it is recommended to place all Flexible Search queries in data access objects. The method names should include the used parameter or uniqueness in their name, like findByCode(String code) or searchUnique(…) to avoid double checks somewhere else in the code. A data access object should never return null for search methods, it is better to return an empty list. This will reduce the risk of null pointer exceptions and follows also the Hybris Commerce practice. The name of a data access object should also include the suffix Dao, e.g. CustomAcmeB2BBudgetDao to find B2BBudgetModel based on various parameter like code.

## 2.4 General Design Pattern

In object-oriented development, it is a common practice to use various design patterns as templates for solutions of recurring problems. Every developer, not only Hybris Commerce developer, has to keep quite a few of them in mind. The most used design patterns in the project should be also documented and agreed upon by the development team. Rather than attempting to apply all known design patterns, it is better to identify and document a small number of them and make sure they are used properly.

Design pattern are grouped in

- Creational Pattern
- Structural Pattern
- Behavioral Pattern
- Concurrency Pattern

The development team should agree to some focused design patterns for the project and document this.

More detailed description about design pattern can be found on the internet, like here, or for example in the book "Design Patterns: Elements of Reusable Object-Oriented Software" from "Gang of Four".

### Where to See Examples of Those Principles

The Hybris Commerce Accelerator are an example implementation of an e-commerce website with the usage of the above-mentioned design principles of Facades, Services, and DAO objects. It shows also the usage of DTO objects in the presentation layer.

# 3 Code Style

## 3.1 Coding Principles

It is highly recommended to define and document some basic coding principles, like the examples below. This can be easily documented as a document or Wiki page, and also be a base guideline for every code review.
Use of NULL (click here to expand...)

## 3.2 Use of Null (Example of Coding Principle)

Hybris Commerce avoids the use of NULL as a return value in most service implementations. It favors throwing runtime exceptions if a precondition is not satisfied or a result is not expected. The use of NULL as return value can cause NullPointerExceptions during the code execution and also adds unnecessary null checks to the code.
**Example: de.hybris.platform.product.impl.DefaultProductService**

```
public ProductModel getProductForCode(final String code)
{
  // This might throw a IllegalArgumentException
  validateParameterNotNull(code, "Parameter code must not be null");

  final List<ProductModel> products = productDao.findProductsByCode(code);

  // This might throw a UnknownIdentifierException or AmbiguousIdentifierException
  validateIfSingleResult(products, format("Product with code '%s' not found!", code),
  format("Product code '%s' is not unique, %d products found!", code,
Integer.valueOf(products.size())));

  return products.get(0);
}
```

### 3.2.1 Use of Null in Our Own Code

- The use of null leads to null pointers. Avoid the use of null anywhere in your code.
- A method should never return null.
- Avoid null checks of objects you obtained from a method call in your own code. Null should never be returned.

The following example shows a common requirement for a DAO method to find an object while avoiding the use of null. This example makes use of the Guava library, but Java 8 provides the Optional<> generic class as well.

The result of the findOrderById method is either "found something" or "found nothing", and nothing is an expected scenario.

Throwing an exception in case nothing was found will cause performance issues if the method is called very often and no result is a frequent result (see why)

```
public Optional<Order> findOrderById(String id)
{
   OrderResponse response = lookup(id)
   if (response.success)
      return Optional.of(response.result);
   else
      return Optional.absent();
}
```

Handling of the result in the client code:

In our case, result and no result are expected business cases!

```
Optional<Order> result = findOrderById(id)
if(result.isPresent()) {
   // result logic using result.get()
} else {
   // no result logic
}
```

The Java 8 lambda syntax provides an interesting way to handle the results without the if () {} else {} pattern:

```
// assuming we want to calculate the order if present and do nothing otherwise:
findOrderById(id).ifPresent( o -> calculationService.calculate(o); );

// or if we want to return the entries if present and an empty list otherwise
return findOrderById(id).map( o-> getEntries() ).orElse(Collections.emptyList());
```

In case the no result case is an exceptional case which cannot be recovered from you can throw a runtime exception. No special handling is then required in the client code.

# 4 Exception Handling

## 4.1 Exception Handling (Example of Coding Principle)

- Use exceptions for exceptional cases that you do not expect. If you expect something to happen to handle it by return value.
- Don't use checked exceptions unless you have a very good reason for it!
- Define Exception classes for the various error cases you want to handle through Exceptions (it is easier to handle an OrderNotFoundException and display a nice error message than a plain BusinessException)
- When you have to catch an exception that you don't know how to handle, simply wrap it in a RuntimeException and bubble it up. Never swallow any exceptions. If you need to ignore an exception, comment why you need to and add at least a log statement.
- Use exceptions to express error codes
- don't let your function return an int error code
- don't let your function return a complex type to contain error attributes

- remember exception types can be complex but think of situations where you might need serialization
- If you decide to throw RuntimeExceptions in your code, this needs to be documented with a @throws declaration in your interface documentation.

### 4.1.1 Validation of Input Parameters

- Use de.hybris.platform.servicelayer.util.ServicesUtil.validateParameterNotNull,or Guava preconditions, or org.apache.commons.lang3.Validate or org.springframework.util.Assert classes to check if an input parameter was provided.
- Throw a runtime exception in case of a wrong method usage (fail as early as possible)

### 4.1.2 Validation of Results

- always return results that are valid
- Throw a runtime exception in case of an invalid data state
- Do not throw exceptions for results that are to be expected.

# 5 Code Style

## 5.1 Optimizations vs. Readability

The coding principles should also define the dominant distinction between readability and performance. Some developers tend to place a lot of functionality in one line, making the code hard to read and debug just to have an irrelevant performance gain. The definition of this principle should also recommend/define a certain depth of modularity or preferred technologies, like

- development against interfaces
- external bean vs. internal or "hard-wired" object binding
- usage of Spring annotation vs. XML based Spring wiring
- AOP vs. inheritance for behaviour/implementation changes

Example:

| | | **Readability** |
|---|---|---|
| Example 1 | original | new DoSomethingObject(new SelectionContext(context, map), currentUser.getCurrentRole().getRole_id()).execute(getResultContext()); |
| | better | new DoSomethingObject(<br>new SelectionContext(context, map),<br>currentUser.getCurrentRole().getRole_id()<br>).execute(getResultContext()); |

| | | **Readability** |
|---|---|---|
| | better | final SelectionContext selectionContext = new SelectionContext(context, map);<br>final String roleId = currentUser.getCurrentRole().getRole_id();<br>final DoSomethingObject doSomethingCmd = new DoSomethingObject(selectionContext,roleId);<br>doSomethingCmd.execute(getResultContext()); |
| | | |
| Example 2 | original | int fourTimes = originalValue << 2; |
| | better | int fourTimes = originalValue * 4; |

Most of the above shown "optimizations" will be performed at the compiler level today, so it is rarely a good practice to make the code less readable to attempt to manually optimize a piece of code. The principle should be "readability first".

## 5.2 Code formatting

Another more readability and reusability oriented area is the code formatting, this should be standard through all developers to avoid issues with the look & feel and different IDE's and operation systems. The common set of rules for code formatting should cover the following topics

| **Topic** | **things to cover** |
|---|---|
| filename | |
| file encoding | UTF8<br>line end character |
| tabulator & spaces | leading spaces<br>tabulator width<br>spaces before and after round brackets |
| imports | NO wildcard imports<br>sequence of imports |
| braces | line break before and after braces |
| line length | max line length, like 100 characters |

| Topic | things to cover |
|---|---|
| comment /<br>block comments | reasons for comments<br>comments in the general/clean code<br>block comments (max size) |
| variable naming | camel case<br>max name length<br>literals |
| @Override | use of @Override annotation |
| static objects / variable | general usage of static objects and variables<br>what & how can be solved with Spring |

In the table above are some example topics to cover, but it is not limited to them.

## 5.3 Javadoc

Each class as well as each public method, except Getter and Setter methods, should have Javadoc, especially the public method represents an API for other extentions. These Javadoc's should be still accompanied with descriptive method and parameters.

- Avoid the plain usage of automatically ¬generated comments.
- Each protected method should have as well as Javadoc since it is also part of the newly created API.
- The Javadocs should explain how the extendibility is designed.
- Synchronization needs to be documented.
- Don't document property style getters and setter (e.g. on a DTO).
- Use Javadoc to document what a method is doing instead of inline comments.

## 5.4 *-items.xml

The items.xml files contain the various elements of the domain model. These *-items.xml files require an internal structure which needs to followed.

- collectiontypes
- enumtypes
- relations
- itemtypes

This order can't be changed, but the parts which are not used do not need to be included.

### 5.4.1 Guidelines Inside of *-items.xml

- Follow the type system rules from the Type System

- Respect the naming convention (camelcase, singular/plural of attributes/types)
- Provide a description tag for describing attribute. Attribute type size ("will it fit?")
- Set the unique flags for attributes which are part of the business key
- Do not add unnecessary indexes, as index managing is cost intensive. As a rule of thumb, indexes should only be added for attributes used in flexible searches. Please take also the used database in consideration for your index design.
- Attributes which need display in different language must be localized.
- Avoid using collections in favor of relations. Check that each type/attribute provides a basic hmc configuration.
- Define types in order of inheritance. More abstract types need to be defined more to the beginning of the items.xml file and more concrete types need to be defined more to the end.
- Any additions to items.xml must be peer reviewed and then reviewed by lead developer as item.xml is critical part of the application.
- Before adding any itemtype to item.xml, make sure that there is no type with the same name in any of the extensions. Otherwise it must be used/extended to avoid the multiple instances of the same type.

# 6 Tools

Various tools can help during the development process to reduce the amount of “bad code”. It is recommended using tools to make sure that the general and/or project specific coding standards are followed.

These tools will not remove the need of peer review and the verification of standards during the peer review. This an essential task for the architect and lead developer to make sure that these guidelines are followed and implemented.

## 6.1 Development IDE (e.g. Eclipse)

Most common IDE’s today offers the support to achieve various goals, like

- Creating Javadoc’s
- Save Actions for code formatting
- Organize imports

The usage of this functionality will not replace the responsibility of the developer to follow the standards, the IDE can only support the developer in following them.

For Hybris Commerce development itself, no specific IDE is required, the decision to use a specific IDE belongs to the developer and their teams. Commonly used are the following IDE's

- Eclipse
- Spring Tools Suite (Eclipse based, brings a lot of plugins pre-installed for Hybris Commerce development, like Spring bean validation, Ant & Maven support, ...)
- IntelliJ (a lot of OOTB functionality for Hybris Commerce development, Spring bean validation, Ant & Maven support, ...)

## 6.2 PMD

PMD is a tool for static code analysis, like checking the code complexity, possible code issues.

PMD can be used in various IDE's but should be included in the continuous integration build process as well. The integration to the CI build will ensure that every committed code will match the defined standards. The execution at the developer machine is recommended to accelerate the feedback loop though.

The main areas with are verified from PMD are

- Possible bugs ¬ empty try/catch/finally/switch statements.
- Dead code ¬ unused local variables, parameters and private methods.
- Sub-optimal code ¬ wasteful String/StringBuffer usage.
- Overcomplicated expressions, like unnecessary if statements or for loops that could be while loops.
- Duplicate code ¬ copied/pasted code means copied/pasted bugs.

The Hybris Commerce platform provides a set of common PMD rules, which are included in the platform code and can be execute with the ant command (ant pmd).

## 6.3 Sonar

SonarQube, or commonly know just as Sonar, is a tools to visualize the results of static code analysis and is also able to show the change during different times of the project. Sonar should be included in your CI build process at least for every release build, but better in a daily build job, because Sonar will provide an easy overview of the current quality metrics of the project, like

- Code duplications
- Dependency cycles
- Cyclomatic complexity
- Violation against the coding standard

These graphics provide more information then sometimes during a peer review can be catch, like Copy&Paste code.

Since Hybris Commerce 5.x is a Sonar ant task and a configuration set for it part of the default platform and can be execute with the "ant sonar" and "ant sonarsingle" task. You can set the sonar settings in the local.properties.

## 6.4 Findbugs

Findbugs is an Open Source program for static code analysis in the Java byte code. It is available as plugins for IDE's, like Eclipse, IntelliJ, or for build tools and systems, like Jenkins, Hudson, Gradle, Maven.

The analysis of Findbugs is based on a large set of static rules, which can be easily enhanced with new rules. This capabilities of Findbugs makes it easy to integrate in a development environment and to prevent that common code issues will be committed to the code base. The generated report ranks the potential errors on their risk factor.

The Hybris Commerce platform has no out-of-the-box ant task for the execution of FindBugs.

## 6.5 Checkstyle

Checkstyle is another common Open Source tool for static source code analysis which is also available for multiple IDE's and build tools. The performed tests inside of CheckStyle are more focused on the representation of the source code then analyzing the content itself. The most helpful part is the detection of code duplicates in larger code bases.

The Hybris Commerce platform has no out-of-the-box ant task for execution of Checkstyle.

# 7 Source Code Repository Exclusions

The purpose of source code repository exclusion is to enable every developer to have a clean workspace to work with. This will also ensure that only intentionally modified or created files can be committed to the source code repository.

All common source code repositories, such as Git, SVN, support exclusion of files and folders by name or pattern.

| Repository | Description |
|---|---|
| Git | supports global and repository related .gitignore files<br>it is recommended not to use the local global ~/.gitignore_global file, because it is only related to the local machine |
| SVN | has ability to exclude files and folders via the svn property ignore<br>can be performed via the "svn propset svn:ignore" command |

gitignore.example is an example for a typical gitignore file for a Hybris Commerce project, this file can be also used to set the ignore property for a SVN repository or in the source code repository of your choice.

See also gitignore file for specific procedure to make your workspace clean when the provided example is not sufficient.

Steps:

1. Deactivate the Eclipse build
2. Perform an Eclipse clean of all custom extensions
3. Launch an ant clean
4. Perform a commit of the custom extensions
5. Launch an ant all
6. Activate the Eclipse build

7. Synchronize the custom extensions
8. Ignore all new files in the source repository

# Recommendations for Extensions

# 1 Project Specific Extensions

## 1.1 Project-Specific Prefix

In order to separate SAP Hybris default elements (extensions, items, Spring beans) from project-specific elements it is highly advisable to use a special prefix **<project_prefix>**, common for each project extension. In most cases, it's a project acronym, very often provided by the customer directly.

An example of such prefix is **sp** - for *Super Project* project.

## 1.2 Extensions Location

It is good practice that project extensions are placed under /bin/custom/<project_prefix> folder. It later allows to group them in the IDE in a single *working set* and separate from other extensions (platform or team/partner specific etc.) as well as repository structure: very often platform extensions are not uploaded to the repository.

## 1.3 Reduced Template Concept

Hybris extensions are usually generated from the **yempty** template extension. This template contains a lot of elements, not necessarily required. An example of redundant code may be a web-app module with dummy media, a sample controller, libs etc. Consider a template extension as empty and as reduced as possible, with all non-essential elements removed. Its structure is visualized in the diagram below:

```
yvoid
├── extensioninfo.xml
├── extgen.properties
├── project.properties
├── resources
│   └── yvoid-spring.xml
└── src
    └── yvoidpackage
        ├── constants
        │   └── YVoidConstants.java
        └── jalo
            └── YVoidManager.java
```

# 1.4 Core Extensions

Naming convention: **<project_prefix>core**

A core extension is usually generated from **yacceleratorcore** template and not from **yempty** since it contains the web module, which in most cases is not needed. The core extension usually consists of common elements reusable by other extensions, such as:

- items definitions (items.xml)
- classes related to Hybris models: services, strategies, DAOs, validators, interceptors, comparators
- utility/helper classes or other classes strictly related to core functionality and not belonging to other extensions (e.g. custom type exceptions)

## 1.4.1 Type Codes Ranges

Define a range of item **type codes** for each **core extension** and keep track of the already used codes. The easiest solution is to add appropriate comments in every **items.xml** file informing about:

- available type code ranges for this extension
- last type code used

Information regarding type code ranges can be also stored in a wiki page as one central place with all type ranges for each project/team extension. It has the following benefits:

- developers don't have to check type codes in each items.xml file
- avoid conflicts (altering of an item type code requires a manual change in SQL in composedTypes and deployment tables)

# 1.5 Project Data Extension

Different environments (DEV, QA, PROD...) require different sets of data. Project data should be centralized and packed into one dedicated extension. Here we can define two types of project data:

- Essential data - some data is regarded as essential and is always imported
- Customer data - this data is very project specific, required for certain functionality

Provide configuration that allows for the selection of data types that should be imported during system initialization/update. It's very handy to have some kind of data versioning mechanism to provide incremental import (e.g. per sprint or release).

# 1.6 Test Data Extension

Naming convention: **<project_prefix>testdata**

This extension is separated as it is feasible only for test environments and should not be included in the production package. The extension would contain only test data for manual tests and demonstration purposes, automated UI tests and also data for performance tests.

## 1.7 Other Project Extensions

The table below presents three project extensions with their naming conventions and the template they are derived from:

| Name | Naming convention | Template |
|---|---|---|
| Storefront | <project_prefix>storefront | yacceleratorstorefront |
| Facade | <project_prefix>facades | yacceleratorfacades |
| OCC/webservices | <project_prefix>webservices | ycommercewebservice |
| Cockpit | <project_prefix>cockpits | yacceleratorcockpits |

## 1.8 hMC Extension

Naming convention: **<project_prefix>hmc**

The hMC is deprecated in favor of the Backoffice cockpit.  Hence, hMC customization should be separated from other code in an hMC extension.

## 1.9 Backoffice Extension

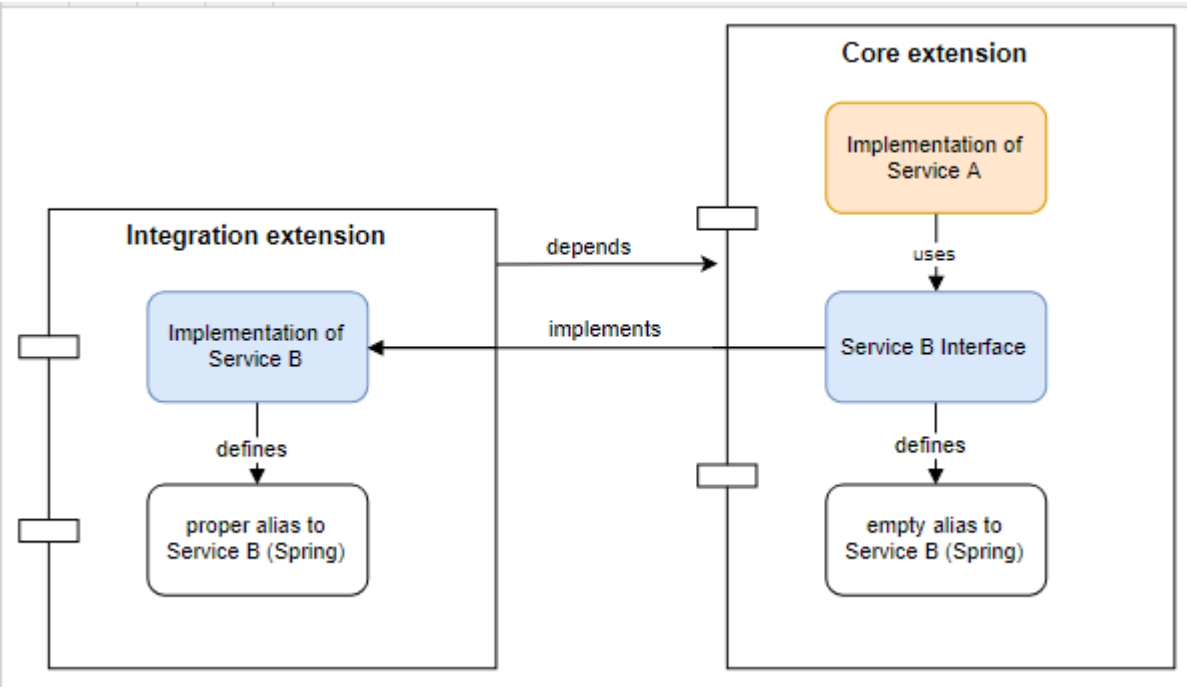
Naming convention: **<project_prefix>backoffice**

The backoffice extension with project specific backoffice cockpit customizations is usually generated from **ybackoffice** template.

Consider having a separate extension for each backoffice perspective/role to avoid having all functionalities in the Administration perspective. Therefore, two approaches are either separation per functionality or per perspective.

## 1.10 Integration Extensions

It is recommended to use one extension for integrations that have many of common parts (for example product and price import may use the same technology and may have many common elements). Integrations with different requirements or technology should be implemented in separate extensions (for example OCC vs DataHub integration points). Interfaces should be defined in the core extension and their implementations in the integration extension in order to avoid circular dependencies:




Test data for integration tests should be a part of the integration extensions.

# 2 IDE Configuration

## 2.1 Working Sets/Module Groups

Working sets (Eclipse) or module groups (IntelliJ IDEA) allow developers to:

- be more efficient by being able to focus on a set of the extensions that are relevant to their tasks, such as project-specific extensions located in bin/custom folder
- disable unused extensions so the whole validation, compilation, and build processes are more efficient

## 2.2 Working Sets in Eclipse

Grouping extensions into working sets can be done either manually or automatically, for example with help of SAP Hybris Commerce Development Tools for Eclipse.

To optimize development work it is recommended to create working sets in Eclipse:

- <project_prefix> exts - Projects in this working set will be modified most frequently. This working set may have different IDE settings (such as Spring Validator and Spring Bean MetaData Builder) which might consume more computer resources.
- Platform & ACC gensrc exts - Projects in this working set should be refreshed after ant clean all.
- Hybris exts - Projects in this working set should not be modified during project development. They are closed, binary projects with no source.

- Cockpit exts - Projects in this working set are related to cockpits. They should be normally closed for performance reasons.

## 2.3 Module Groups in IntelliJ IDEA

IntelliJ IDEA supports so-called **Module Grouping**, which enables separate modules for different extensions groups (hybris extensions, platform and config, custom extensions). It is equivalent to Eclipse working sets.

# 3 Wiki Page with Guidelines

It is a good practice to create a project specific page with:

- guidelines regarding extensions creation, structure, maintenance etc.
- declared project specific prefix
- agreed best practices regarding code review, branching strategy and naming conventions

# References


- https://wiki.hybris.com/display/hybrisALF/Coding+Standards
- https://wiki.hybris.com/display/hybrisALF/Developer+Recommendations+for+Extensions